

Тема 3. Производни класове и наследственост

наследяване – фундаментален механизъм, който позволява създаване на нови класове на база вече създадени; повишава надеждността на програмата.

суперклас (клас, клас-родител) – вече съществуващ клас, който ще се използва за създаването на нов.

производен клас (под-клас, наследяващ клас) – класът, който се създава на база суперкласа. В Java, един производен клас може да се създаде само на база един суперклас (в Java няма множествено наследяване, а само многослойно (многократно) наследяване)

многократно наследяване – когато един производен клас се явява суперклас за друг клас.

Полза: в производния клас не е нужно да се описва кодът на суперкласа, а само „допълнителните“ членове на класа. Производният клас наследява всички нескрити полета и методи, т.е всички полета и метода, които са дефинирани като private не се наследяват!

Дефиниране на производен клас

```
class име_на_производен_клас extends име_на_суперкласа {  
    // описание на производния клас  
}
```

```
class Alpha {  
    int number;  
    void show() {  
        System.out.println("Целочислено поле: "+number);  
    }  
}  
class Beta extends Alpha {  
    char symbol;  
}
```

Обектите, които се създават на база тези два класа са напълно различни. При създаването на обекти от клас Alpha, те ще имат едно поле и един метод. При създаването на обекти от клас Beta те ще имат две полета и един метод.

Пример 1: Създаване на подклас

Да се създаде производен клас, който има още едно целочислено поле, метод, който да визуализира стойността му и метод, който изчислява сумата на двете полета. Да се създаде обект на база производния клас и да се изведе сумата.

Наследяване и private членове

Членовете на суперкласа, които са **private** не се наследяват, това означава че **private** полетата и методите не са достъпни в производния клас, т.е. не можем да се обръщаме директно към тях дори и в самия клас.

Наследяване = достъпност

Технически обаче ненаследените полета в производния клас съществуват! При създаване на обект на база производен клас първо се извиква конструкторът на суперкласа, а след това се изпълняват командите за създаването на обекта на производния клас. Чрез конструктора се заделя място в паметта за всички членове на обекта, в това число и ненаследените!!!!

Пример 2: Наследяване и частни членове

//Суперклас с частни полета:

```
class Alpha2{
private char symbol;
void show(){
    System.out.println("Символ: "+symbol);
}
void set(char s){
    symbol=s;
}
}
```

//Производен клас:

```
class Beta2 extends Alpha2{}
```

//Главен клас:

```
class Primer2{
public static void main(String[] args){
    Beta2 B2=new Beta2();
    B2.set('B');
    B2.show();
}
}
```

Изход в конзолата:

Символ: B

Пакети и protected членове

пакет – контейнер за класове, тяхното пространство от имена. В рамките на пакета името на класа трябва да бъде уникално. В Java в един и същи клас могат да бъдат дефинирани няколко метода с едно и също име. Дефинира се още с първата команда:

package pnpe;

Всеки пакет може да съдържа – класове, интерфейси и подпакети.

За обръщение към клас от друг пакет се използва пълното име на класа: чрез точков синтаксис се изброява цялата йерархия от пакети, където се намира желания клас.

package pnpe.su2.MyClass;

Импортиране на пакети – за да използваме съкратена форма към класовете на външни пакети.

import java.lang.Math;

import pnpe.su2.Myclass;

import pnpe.su2.*;

За използване на статични членове на класа без референция на класа може да бъде извикано статичното импортиране

import static P3.su1.Myclass.number;

import static P3.su1.Myclass.*;

- импортират се само публични класове (дефинирани с public);
- java.lang може да не се импортира;
- името на файла трябва да съвпада с името на публичния клас, ако такъв клас съществува във файла;
- ако в един пакет има няколко публични класа, те трябва да са разположени в различни файлове (в един файл може да има само един публичен клас)

Достъпност на членовете на класа

Място за получаване на достъп	private	protected	public	без спецификатор за достъп
същия клас	+	+	+	+
подклас в същия пакет	-	+	+	+
обикновен клас в същия пакет	-	+	+	+
подклас в друг пакет	-	+	+	-
обикновен клас в друг пакет	-	-	+	-

Конструктори и наследяване

Обектът на производния клас се създава на два етапа – първо се създава частта определена от суперкласа и след това се добавя частта, свързана със самия производен клас. Това е така, защото винаги първо се извиква конструкторът на суперкласа! Особености!

Когато в суперкласа има дефиниран конструктор, който изисква предаване на аргументи, трябва в производния клас да бъде описан конструктор и в тялото на този конструктор да има инструкция за извикване на конструктора на суперкласа (със съответните аргументи). Това става чрез ключовата дума **super**(args), която се поставя като първа инструкция в тялото на конструктора на производния клас. Ако я няма тази ключова дума се извиква конструкторът на суперкласа без аргументи.

```
конструктор (аргументи){  
    //извикване на конструктора на суперкласа  
    super(аргументи);  
    //тяло на конструктора  
}
```

Пример 3: Наследяване и конструктори

```
//Суперклас:  
class Alpha3{  
    int number;  
    void show(){  
        System.out.println("Поле number: "+number);  
    }  
  
    Alpha3(){  
        number=1;  
        show();  
    }  
  
    Alpha3(int n){  
        number=n;  
        show();  
    }  
}  
  
//Производен клас:  
class Beta3 extends Alpha3{  
    char symbol;
```

Платформено независими програмни езици
Семинарно упражнение 3

```
void display(){
    System.out.println("Поле symbol: "+symbol);
}

Beta3(){
    // Извикване на конструктора на суперкласа:
    super();
    symbol='B';
    display();
}

Beta3(int n,char s){
    // Извикване на конструктора на суперкласа:
    super(n);
    symbol=s;
    display();
}
}
}

//Главен клас:
class Primer3{
public static void main(String[] args){
    System.out.println("Първи обект:");
    Beta3 A=new Beta3();
    System.out.println("Втори обект:");
    Beta3 B=new Beta3(2,'A');
}
}
```

Изход в конзолата:

Първи обект:

Поле number: 1

Поле symbol: B

Втори обект:

Поле number: 2

Поле symbol: A

Предефиниране на методи

Ситуация, при която на **наследен** от суперкласа метод трябва да бъде променена функционалността, НО не се променят нито аргументите, нито типа на връщания резултат. В зависимост от къде се достъпва класа се извиква съответната му версия. Ако извикаме метода от обект на суперкласа, ще бъде изпълнена неговата оригинална функционалност. В този случай, за да бъде извикана версията на метода в суперкласа, пред инструкцията за извикване на метода се поставя ключовата дума **super** и се използва точков синтаксис. Ако извикаме метода от обект на производния клас, ще бъде изпълнена неговата предефинирана функционалност. Предефинирането може да се използва едновременно с презареждането.

ключова дума **final** – използва се в суперкласа, за да покаже, че метод не може да се предефинира;

ключова дума **@Override** – анотация, която може да се постави (но не е задължително) пред предефинирания метод, за да се информира компилатора за това, че методът е предефиниран

Принципна разлика между презареждане и предефиниране! При презареждане методите имат еднакви имена, но различни аргументи. При предефинирането методите имат еднакви имена, но и еднакъв тип както на резултата, така и на аргументите.

Виртуалност на метода – версията на метода се определя от класа на обекта, от който се извиква метода!

Пример 4: Предефиниране на метод

```
//Суперклас:
class Alpha4{
private int number;
void show(){
    System.out.println("Поле number: "+number);
}

void set(int n){
    number=n;
}
// Конструктор:
Alpha4(int n){
    set(n);
}
}
```

Платформено независими програмни езици
Семинарно упражнение 3

```
//Подклас:
class Beta4 extends Alpha4{
private char symbol;
void show(){
    // Извикване на версията на метода от суперкласа:
    super.show();
    System.out.println("Поле symbol: "+symbol);
}

void set(int n,char s){
    set(n);
    symbol=s;
}
// Конструктор:
Beta4(int n,char s){
    // Извикване на конструктора на суперкласа:
    super(n);
    symbol=s;
}
}
//Главен клас:
class Primer4{
public static void main(String[] args){
    Alpha4 A=new Alpha4(5);
    A.show();
    A.set(10);
    A.show();
    Beta4 B=new Beta4(10,'Y');
    B.show();
    B.set(100,'X');
    B.show();
    B.set(3);
    B.show();
}
}
```

Изход в конзолата:

Поле number: 5

Поле number: 10

Поле number: 10

Поле symbol: Y

Поле number: 100

Поле symbol: X

Поле number: 3

Поле symbol: X

Заместване на полета при наследяване

При наследяване може да възникне ситуация, при която имената на наследено в производния клас поле съвпада с името на полето декларирано в суперкласа. За да се различават двете полета, тогава когато се обръщаме към полето на суперкласа се поставя ключовата дума **super** и се използва точков синтаксис.

Задачи за самостоятелна работа:

Задача 1: Дефинирайте клас `Person` с полета "собствено име" и "фамилно име". Дефинирайте клас `Student`, наследяващ `Person`, който има свойство "оценка". Дефинирайте клас `Worker`, наследяващ `Person`, със свойства "надница" и "изработени часове". Имплементирайте и метод "изчисли надница за 1 час", който смята колко получава работникът за 1 час работа, на базата на надницата и изработените часове. Напишете съответните конструктори и методи за достъп до полетата (свойства).

Задача 2: Дефинирайте клас `Shape2D`, описващ 2D фигури, който показва размерите на фигурата. Дефинирайте клас `Shape3D`, наследяващ `Shape2D`, който има описва фигури в 3D пространството и показва техните размери. Напишете съответните конструктори и методи за достъп до полетата (свойства).